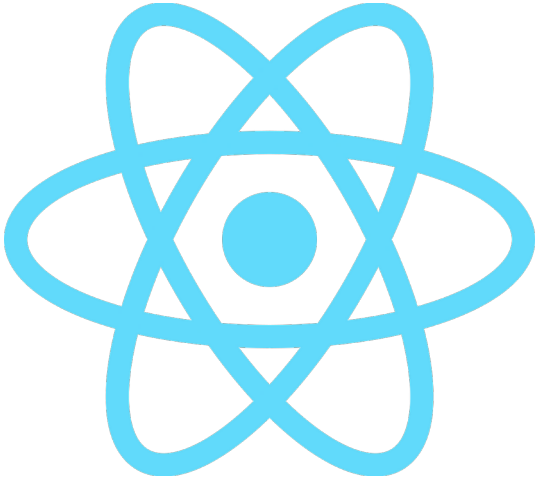
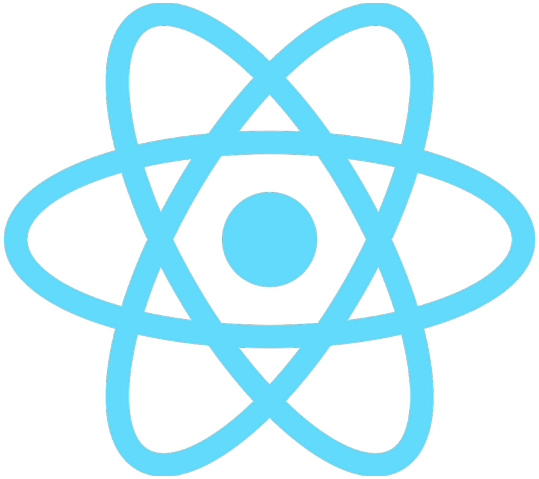


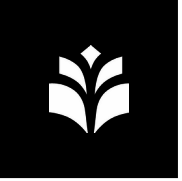
React

From UI problems to component thinking



React

Student is able to
analyze user interface requirements and design reusable, component-based web applications using React



Contents

FROM UI PROBLEMS TO COMPONENT THINKING

- Why do traditional DOM approaches become difficult as apps grow?
- How can a UI be decomposed into reusable components?
- How does information move in a component-based application?

REACT BASICS

Why React?

- Why React, not plain JavaScript? What are React's main characteristics?

Components and JSX

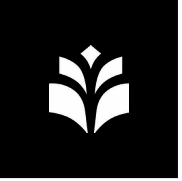
- What is a React component, and how is it defined? What is JSX?

Starting a React project

- How to set up a React project? How to create and use components?

React Developer Tools — inspecting your React application

- How to install React Developer Tools in the browser?
- How to see the component tree of my React application?



FROM UI PROBLEMS TO COMPONENT THINKING

Why do traditional DOM approaches become difficult as apps grow?

How can a UI be decomposed into reusable components?

How does information move in a component-based application?

REACT BASICS

Why React?

Why React, not plain JavaScript? What are React's main characteristics?

Components and JSX

What is a React component, and how is it defined? What is JSX?

Starting a React project

How to set up a React project? How to create and use components?

React Developer Tools — inspecting your React application

How to install React Developer Tools in the browser?

How to see the component tree of my React application?

What we learned in JavaScript

- Manipulate the DOM directly
  `getElementById, addEventListener, querySelector`
- Build simple interactive elements – buttons, forms, and menus
- Handle dynamic data – update content on user actions

```
const btn = document.getElementById('myButton');
btn.addEventListener('click', () => {
  document.getElementById('content')
    .innerText = 'Hello, World!';
});
```

DOM manipulation gets difficult as apps grow

- More UI elements to manage and update
- Hard to track which DOM parts need updating
- State management across the app becomes error-prone
- Code duplication — no structure for reusable parts (buttons, forms, cards)

Result: "DOM spaghetti" — hard to read, test, and maintain

What happens when requirements grow

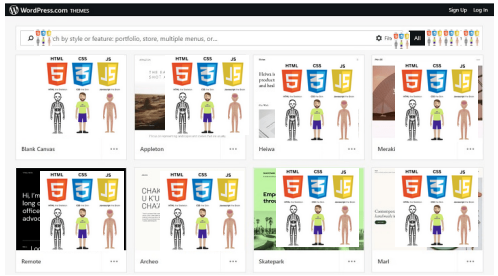
Kažigram — started simple → Show posts, add post, like, delete

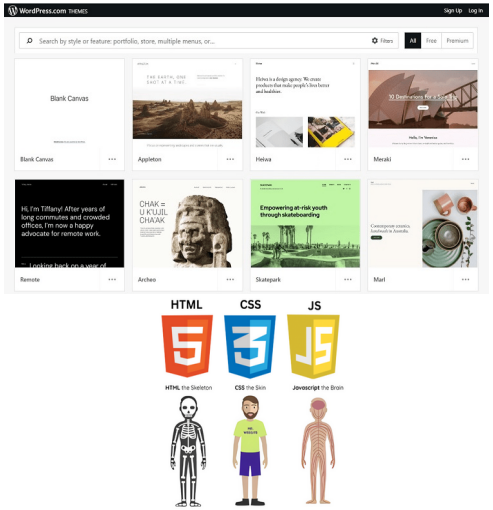
What if we add
→ Comments on each post
→ User profiles
→ Notifications
→ Search and filter
→ Multiple pages (feed, profile, settings)

Vanilla JS approach: each new feature adds more getElementById, addEventListener, innerHTML...
→ Code grows uncontrollably
→ Same logic repeated in multiple places
→ One change breaks something else

UI as reusable components

- solution: split the UI into independent, reusable parts
- Each component:
  → is devoted to one specific task
  → receives data as input (props)
  → returns a user interface
- Direction web development has taken:
  Web Components, Angular, React, Vue.js
- In JavaScript it is very easy to create spaghetti
  — better to structure applications in modules





From requirements to component design

Process:
1. Read the requirements
2. Find repeating patterns → component candidate
3. Identify what data each part needs → props candidates
4. Identify what changes over time → state candidates
5. Draw the component tree

Ask yourself:
→ What repeats? (e.g., each post looks the same)
→ What changes? (e.g., like count updates)
→ What is separate? (e.g., adding a post is its own feature)
→ Who needs this data? (decides where state lives)

Example: Kažigram — requirements

Features:

- Display a feed of cat posts: each post shows: photo, title, caption, like count
- Users can like a post: like count updates immediately
- Users can add a new post: fill in title, caption, and choose the photo
- Users can delete a post

Interface:

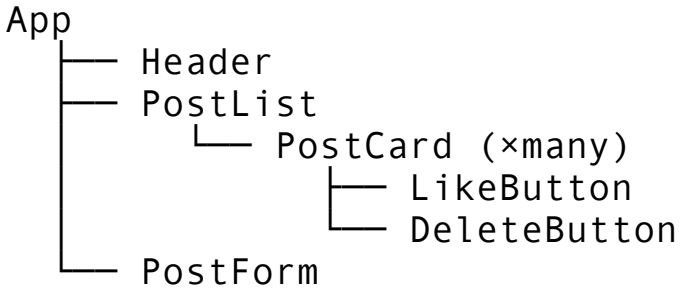
- One main view showing all posts
- A form for adding new posts
- Navigation between the feed and the form

Example: Identifying components

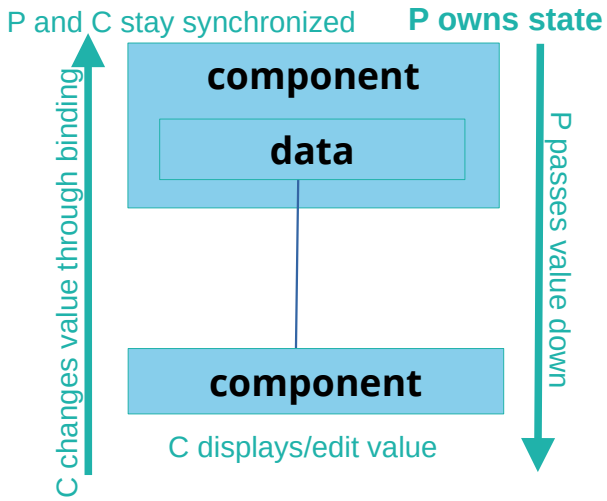
App, the root, holds all posts
Header with logo, navigation
PostList, renders all PostCard components
PostCard (×many) with photo, title, caption, like count
LikeButton (clicks update count)
DeleteButton (removes post)
PostForm, form for adding a new post, with title, caption, image URL inputs

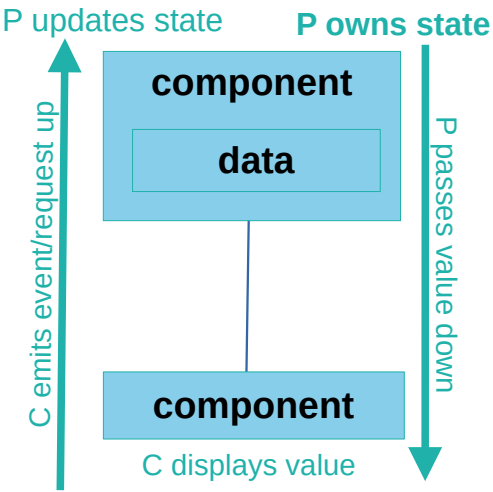
Props and state:
PostCard receives: post object (props)
LikeButton: likes count — component state or App state?
→ Does like count need to be saved? → App state
→ Is it only visual feedback? → component state
PostForm: input field values → component state

There is no single right answer — design decisions matter.



Data binding — two approaches

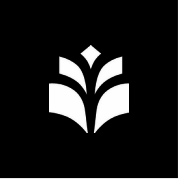




Two-way data binding – data flows in both directions
→ Application updates UI
→ UI automatically updates application data
→ used by Vue (when using v-model). Angular

One-way data binding – data flows in one direction only
→ Application updates UI
→ UI cannot directly update application data
→ used by Vue (via props), React

Both solve the same problem: keeping UI in sync with data. Parent owns state in both cases; the difference is whether the child can only request changes or can sync changes back through binding.



FROM UI PROBLEMS TO COMPONENT THINKING

- Why do traditional DOM approaches become difficult as apps grow?
- How can a UI be decomposed into reusable components?
- How does information move in a component-based application?

REACT BASICS

Why React?

- Why React, not plain JavaScript? What are React's main characteristics?

Components and JSX

- What is a React component, and how is it defined? What is JSX?

Starting a React project

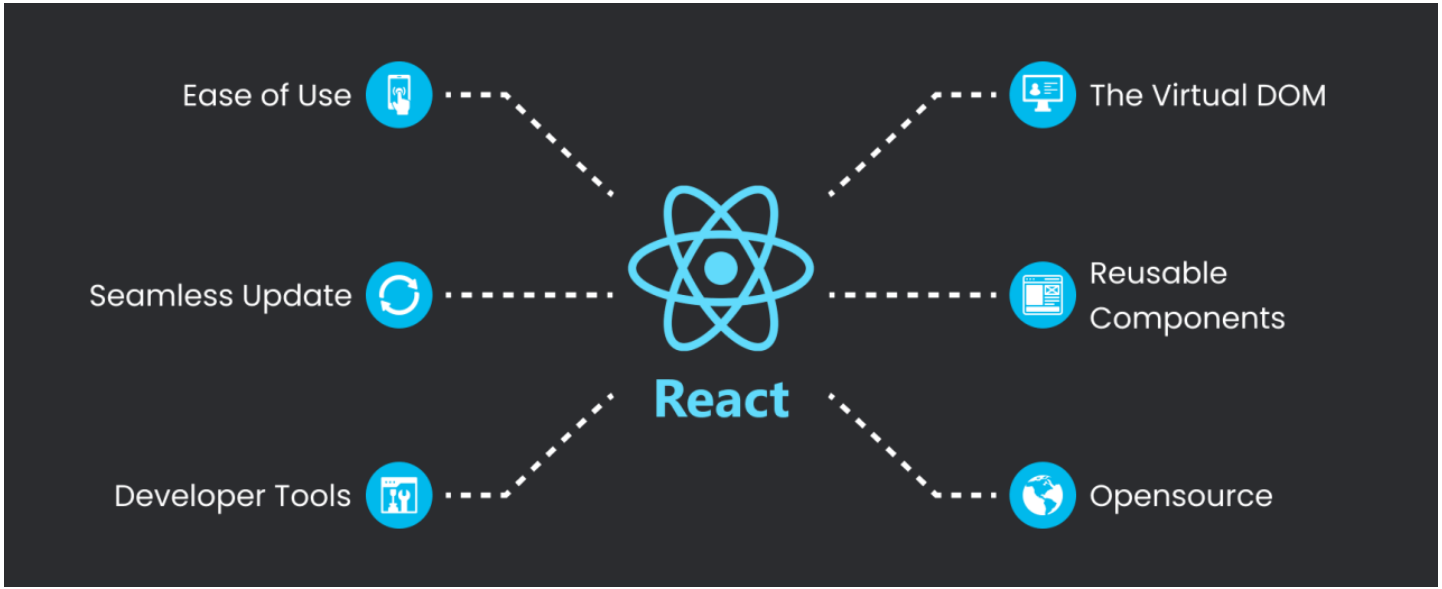
- How to set up a React project? How to create and use components?

React Developer Tools — inspecting your React application

- How to install React Developer Tools in the browser?
- How to see the component tree of my React application?

React – a JavaScript library

A JavaScript library for building web and native user interfaces.



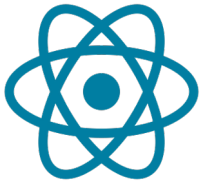
Popular front-end JS frameworks

Angular (2010/2016)
full framework, runtime, TypeScript-first

React (2013)
component library, runtime virtual DOM

Vue (2014)
lightweight framework, runtime

Svelte (2016)
compiler-based framework, no virtual DOM needed at runtime



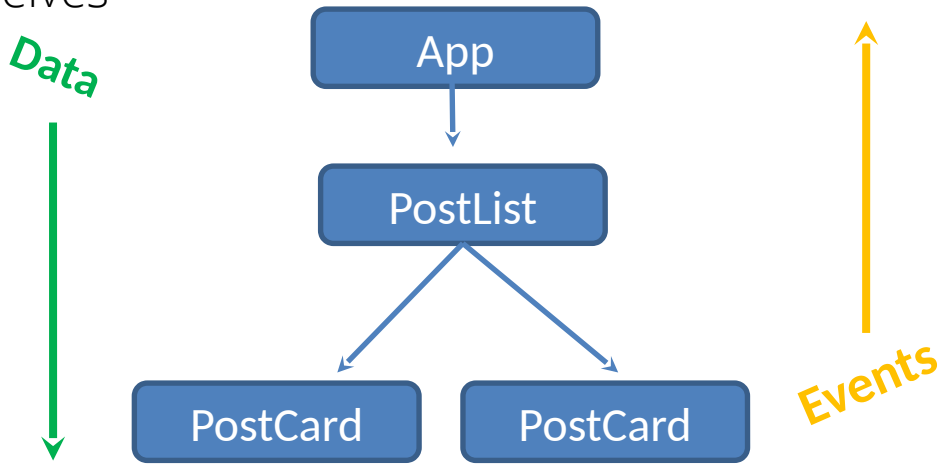
Data flows one way — predictable

In one-way data binding data flows down, events flow up
→ predictable, easier to debug?

Data (props) flow from parent to child: App → PostList → PostCard.
Parent decides what data each child receives

Events flow from child to parent:
PostCard → PostList → App.

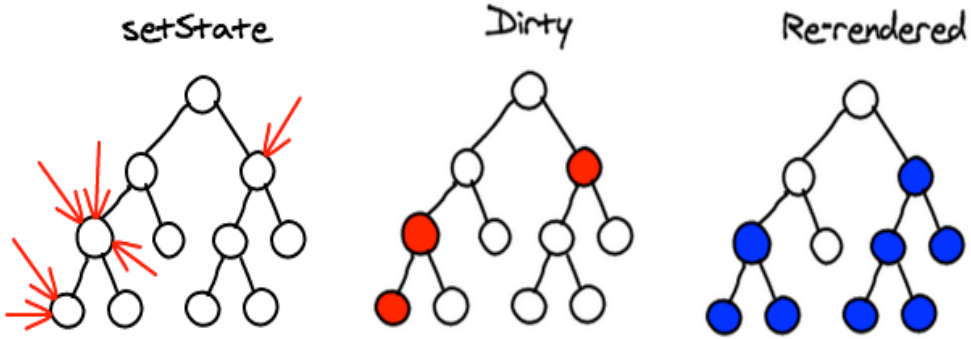
Child notifies parent when something happens (e.g., "user clicked like")
— the parent decides what changes.



Virtual DOM — efficient UI updates

Without React (vanilla JS) developer decides manually which DOM element to update → Works for simple apps, but becomes error-prone as complexity grows

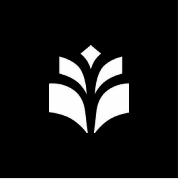
With React developer describes what the UI should look like. React compares new vs. previous Virtual DOM (diffing)React automatically updates only the necessary parts



→ developer does not need to track which element changed

React’s core ideas

- Reusable components → less duplication
  Build once, use everywhere
- Virtual DOM → efficient updates
  React updates only what changed, not the entire page
- One-way data binding → predictable data flow
  Data flows down, events flow up



FROM UI PROBLEMS TO COMPONENT THINKING

- Why do traditional DOM approaches become difficult as apps grow?
- How can a UI be decomposed into reusable components?
- How does information move in a component-based application?

REACT BASICS

Why React?

- Why React, not plain JavaScript? What are React's main characteristics?

Components and JSX

- What is a React component, and how is it defined? What is JSX?

Starting a React project

- How to set up a React project? How to create and use components?

React Developer Tools — inspecting your React application

- How to install React Developer Tools in the browser?
- How to see the component tree of my React application?

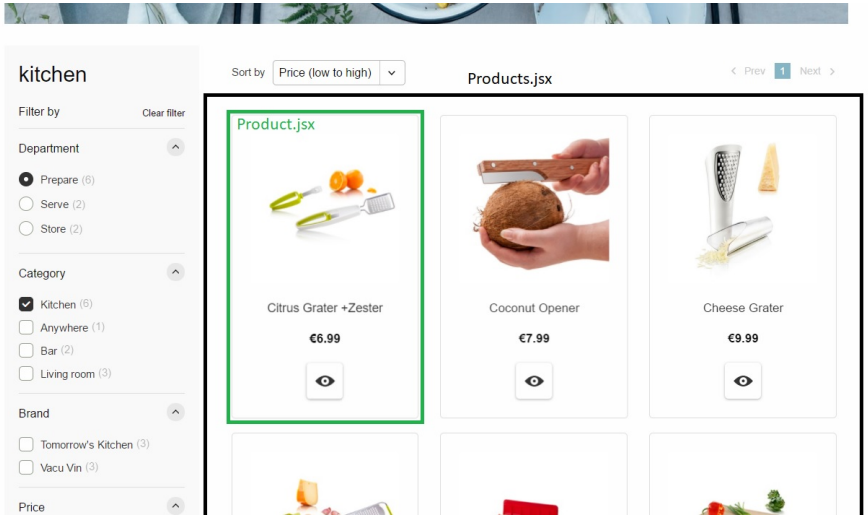
A component as a UI building block

In React, a component

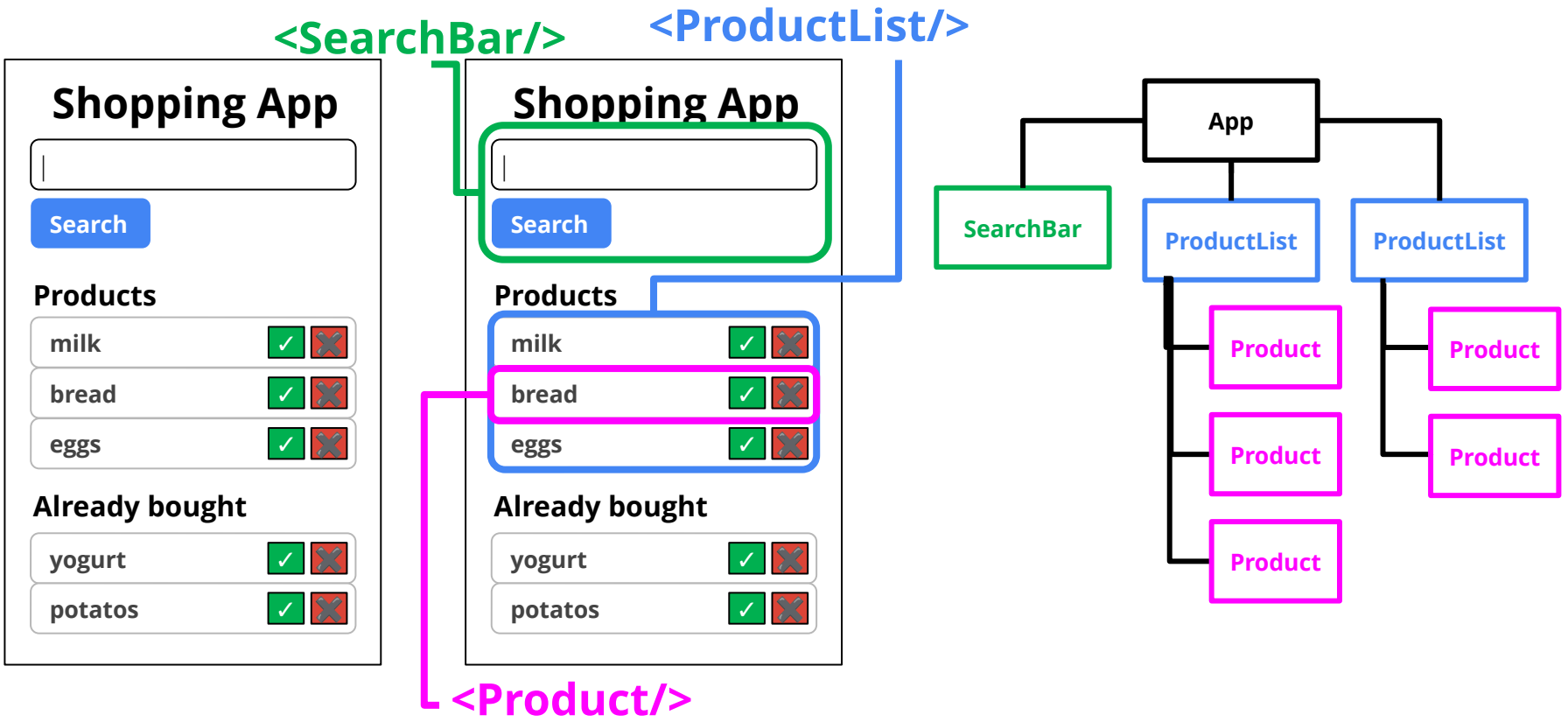
→ is a function that returns the UI as JSX (JavaScript XML)

→ receives data as input (props)

→ is independent and reusable



Example: Shopping App



JSX combines JS and XML

JSX = JavaScript + XML.

HTML is a specific version of XML with fixed tags: , <p>, <button>, <form>

XML (eXtensible Markup Language): a markup language for structured data — like HTML, but you can define your own tags: <book>, <cat>, <PostCard>, <LikeButton>

JSX borrows XML's syntax because in React, you write your own tags (components): <LikeButton />, <App />.

These tags can have a name, attributes, and children — but only the name is required. When a tag has no children, you close it immediately with a forward slash.

```
<MyButton color="blue" shadowSize={2}>
  Click Me
</MyButton>
```

Defining a component — function returning JSX

A component is a function that
→ accepts props as input
→ returns a React element (JSX)

Both traditional and arrow function syntax work, still arrow functions are more common in modern React.

```
// Traditional function syntax
function Welcome(props) {
  return <h1>Hello, {props.name}
</h1>;
}

// Arrow function syntax
const Welcome = (props) => {
  return <h1>Hello, {props.name}
</h1>;
};
```

Babel transpiles to function Welcome(props) {
  return React.createElement(
    'h1',   // element type
    null,   // props (here null)
    'Hello, ',   // child 1 — text
    props.name   // child 2 — variable
  );
}

Note: Class components also exist but are no longer recommended.

Component example: Greeting.jsx

- A component is only a function that returns an HTML to render
- The React element to render can be one or several HTML tags (by example a div or an h1) or other React components
- They are not just templates: they can have multiple methods and different features .

Greeting.jsx

```jsx
import React from 'react';

function Greeting(props) {
  return (
    <div>
      <h1>Hello, {props.name}!</h1>
      <p>Welcome to React.</p>
    </div>
  );
}

export default Greeting;
```

usage from another component

```jsx
<Greeting name="Toast" />
```

result

```jsx
<div>
  <h1>Hello, Toast!</h1>
  <p>Welcome to React.</p>
</div>
```

Hello, Toast!

Welcome to React.

JSX syntax: tags

Component names start with uppercase:
ShoppingCart, CustomButton, PostCard
→ sets them apart from HTML tags (always lowercase)

```
<MyButton color="blue" shadowSize={2}>
  Click Me
</MyButton>
```

```
<div className="sidebar" />
```

↑

Note: class → className in JSX
(class is a reserved word in JavaScript)

JSX as custom HTML elements

JSX defines components as custom HTML elements: custom tags (component name), custom attributes (props), custom children (HTML tags or other React elements)

```jsx
// MyButton component – uses color, shadowSize and fontSize as props
function MyButton(props) {
  return (
    <button
      style={{
        color: props.color,
        boxShadow: `0 0 ${props.shadowSize}px gray`,
        fontSize: props.fontSize
      }}
    >
      {props.children}
    </button>
  );
}
```

```jsx
// Usage of MyButton component:
<MyButton color="blue" shadowSize={2} fontSize="12px">
  Click Me
</MyButton>


<!-- Compare to standard HTML button -->
<button style="color: blue; box-shadow: 0 0 2px gray; font-size: 12px;">
  Click Me
</button>
```

Using JavaScript inside JSX

Any JavaScript expression can be used inside { }

```
function Greeting() {
  const name = "Kinsky";
  const isAdmin = true;

  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>Posts: {42 + 8}</p>
      {isAdmin && <span>Admin</span>}
    </div>
  );
}
```

{ } → expression, not a statement

variables, calculations, function calls, ternary operators

Styling in React — three options

Option 1: External CSS file (same as before)

```
import './App.css';
<div className="card">...</div>
```

Note: className, not class (class is a reserved word in JavaScript)

Option 2: Inline style as a JavaScript object

```
const style = { backgroundColor: '#f0f0f0', color: '#333' };
<div style={style}>...</div>
```

Note: camelCase (backgroundColor, not background-color) because it is a JavaScript object, not CSS

Option 3: Direct inline (double curly braces)

```
<div style={{ backgroundColor: '#f0f0f0', color: '#333' }}>
```

outer { } = JSX expression, inner { } = JavaScript object

Conditional styling

Show or hide elements based on props:

```
function SimpleBox(props) {
  return (
    <div style={{ display: props.show ? 'block' : 'none' }}>
      <p>This box is visible based on the "show" prop!</p>
    </div>
  );
}

// Usage:
<SimpleBox show={true} />  → visible
<SimpleBox show={false} /> → hidden
```

condition ? valueIfTrue : valueIfFalse

Note: double {{ }} — outer: JSX expression, inner: JS object

Conditional rendering

```jsx
function Welcome(props) {
  return (
    <div>
      {props.loggedIn ? (
        <h1>Welcome back, user!</h1>
      ) : (
        <h1>Please log in to continue.</h1>
      )}
    </div>
  );
}
```

condition ?
valueIfTrue
:
valueIfFalse

```jsx
<Welcome loggedIn={true} />
```

Welcome back, user!

```jsx
<Welcome loggedIn={false} />
```

Please log in to continue.

JSX is transpiled to JavaScript by Babel

Browsers can only execute vanilla JavaScript — not JSX.
Transpilation converts JSX to vanilla JS.

```
// What we write (JSX):
function Hello() {
  return <span>Hello!</span>;
}

// transpiles to (what Babel produces, vanilla JS):
function Hello() {
  return React.createElement('span', null, 'Hello!');
}
```

This happens automatically in the background
→ we never need to write React.createElement() ourselves

Examples: JSX to JavaScript by Babel

```jsx
<span>Hello!</span> // JSX example without props

React.createElement( // Babel produces
  'span',       // 1. element type: span
  null,         // 2. props: none (null)
  'Hello!'      // 3. children: text content
)

<MyButton color="blue">Click Me</MyButton> // JSX example with props

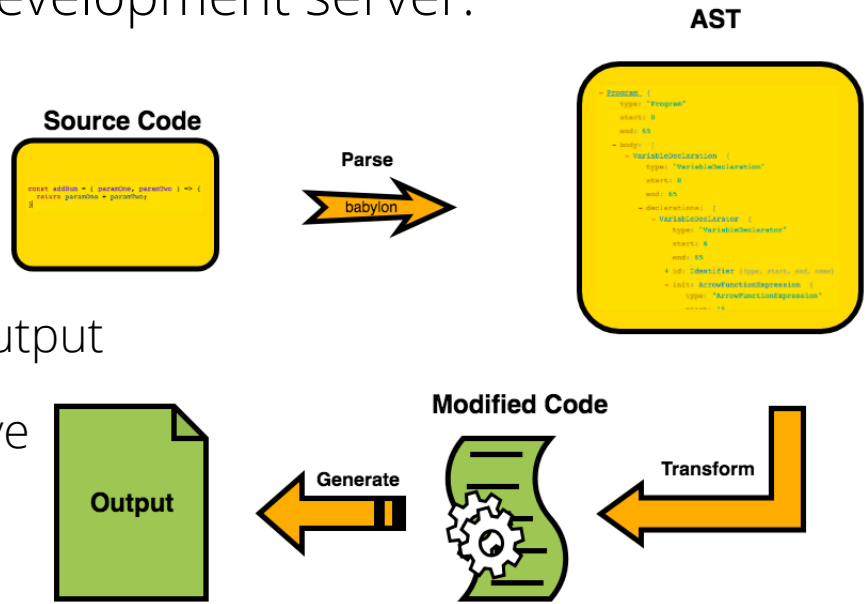
React.createElement( // Babel produces
  MyButton,             // 1. element type: MyButton component
  { color: "blue" },    // 2. props: { color: "blue" }
  "Click Me"            // 3. children: text content
)
```

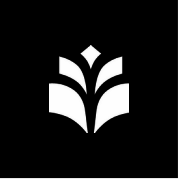
Babel as the transpiler

Babel converts JSX to standard JS behind the scenes.
To do this automatically, we need a development server.

Steps:
1. Parse — read JSX source code into an Abstract Syntax Tree (AST)
2. Transform — modify the AST
3. Generate — produce vanilla JavaScript output

With Vite this happens automatically on save
→ no manual configuration needed





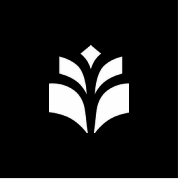
Summarizing components and JSX

a component = JavaScript function returning UI as JSX.

JSX = XML-like syntax inside JavaScript

JSX tags: uppercase = React component, lowercase = HTML

Babel transpiles JSX to vanilla JavaScript



FROM UI PROBLEMS TO COMPONENT THINKING

- Why do traditional DOM approaches become difficult as apps grow?
- How can a UI be decomposed into reusable components?
- How does information move in a component-based application?

REACT BASICS

Why React?

- Why React, not plain JavaScript? What are React's main characteristics?

Components and JSX

- What is a React component, and how is it defined? What is JSX?

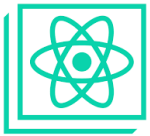
Starting a React project

- How to set up a React project? How to create and use components?

React Developer Tools — inspecting your React application

- How to install React Developer Tools in the browser?
- How to see the component tree of my React application?

Tools for starting



Old-fashioned (deprecated)

create-react-app — no longer recommended



Modern (**frontend** only) – we'll use this

vite https://vitejs.dev → fast dev server, HMR, production builds



Modern (**fullstack** = frontend + backend)

Next.js — when you need server-side rendering

Starting a React project with Vite

Install Node.js first: https://nodejs.org/en
Node.js comes with npm (package manager)



```
$ npm create vite@latest
Need to install the following packages:
create-vite@9.1.1
Ok to proceed? (y)
- -
```

Answers to the prompts:

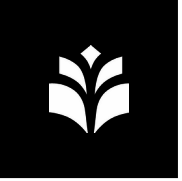
```
Project name: my-app (name of your project)
Package name: accept the default (press Enter)
Select a framework: React
Select a variant: JavaScript
Which linter to use: ESLint
Install with npm and start now? Yes
- -
```

```
◇  Starting dev server...

> my-app@0.0.0 dev
> vite

  VITE v8.1.5  ready in 226 ms

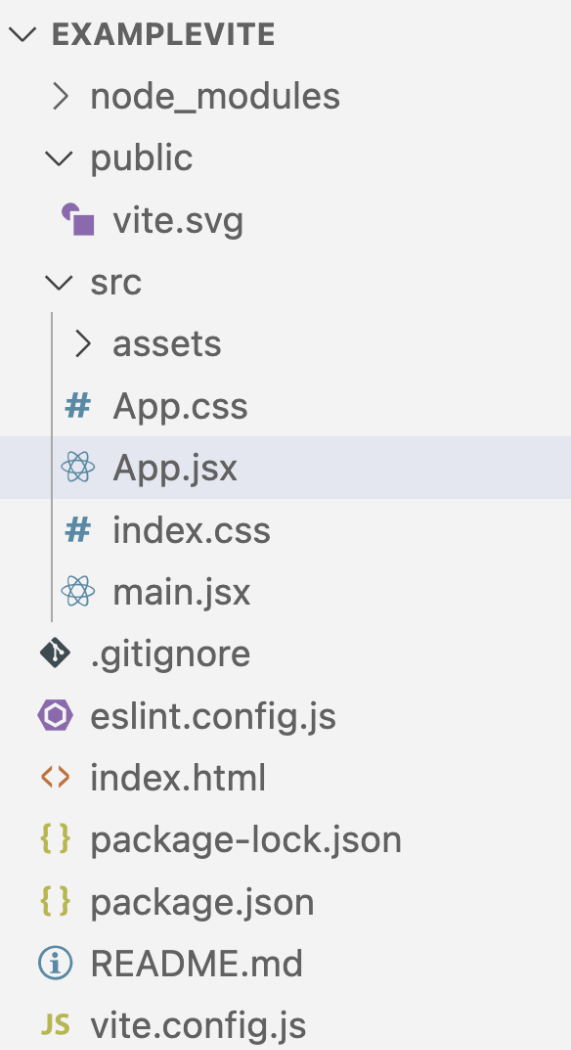
  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```



Project structure

The tool generates the files you need to build you app

- public/ → static files (vite.svg)
- src/ → your code here
  - App.jsx → main component (hello-world included)
  - main.jsx → React entry point
  - App.css → component styles
  - index.css → global styles
- .gitignore → Git lecture: what not to commit
- index.html → only HTML file we need
- package.json → dependencies and scripts
- vite.config.js → development configuration
- README.md → project description



package.json — dependencies & scripts

npm run dev
starts the dev server

npm run build
creates production files

npm run lint
checks code quality

The dependencies used in production, e.g. the React library and dependency connecting the React to the browser DOM

The dependencies used only during development, vite → dev server + build tool, eslint → code quality checking, @vitejs/plugin-react → enables JSX in Vite

```
{
  "name": "examplevite",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  },
  "devDependencies": {
    "@eslint/js": "^9.11.1",
    "@types/react": "^18.3.10",
    "@types/react-dom": "^18.3.0",
    "@vitejs/plugin-react": "^4.3.2",
    "eslint": "^9.11.1",
    "eslint-plugin-react": "^7.37.0",
    "eslint-plugin-react-hooks": "^5.1.0-rc.0",
    "eslint-plugin-react-refresh": "^0.4.12",
    "globals": "^15.9.0",
    "vite": "^5.4.8"
  }
}
```

npm run dev — development server

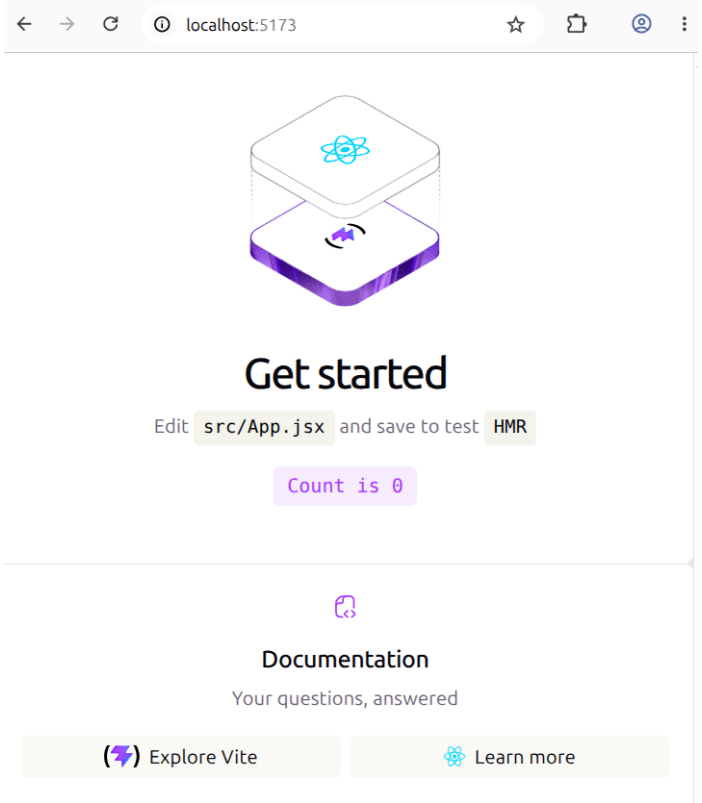
Run `npm run dev` on project folder to launch the dev server.

Open the browser at http://localhost:5173/ (if not opened automatically)

You can now edit src/App.jsx and see changes immediately.

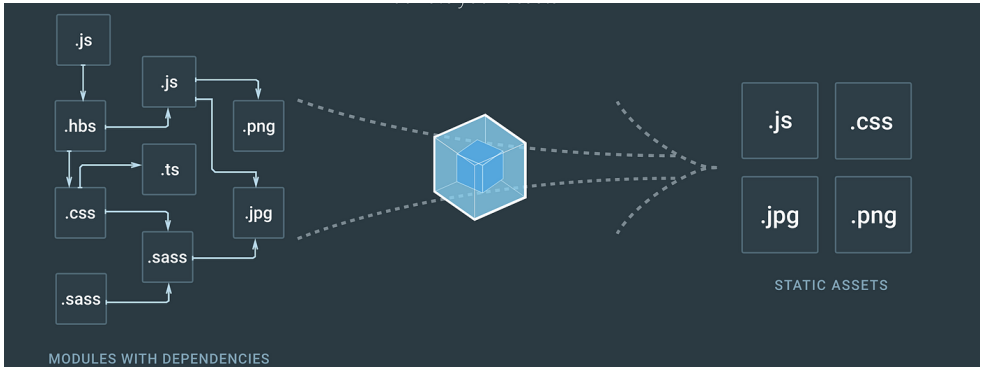
```
VITE v8.1.5  ready in 226 ms

➜  Local:   http://localhost:5173/
➜  Network: use --host to expose
➜  press h + enter to show help
```



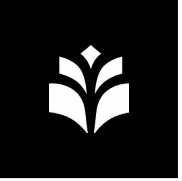
HMR – Hot Module Replacement

HMR swaps, adds, or removes modules while the app is running — without reloading the page.



When a file is saved:
→ Dev server detects the change
→ Only the changed module is recreated
→ Browser updates instantly

Result: faster development — you see changes immediately without losing the current application state.



Creating and using a component

1. Create a new file in src/: Greeting.jsx

```
function Greeting(props) {
  return <h1>Hello, {props.name}</h1>;
}
export default Greeting;
```

2. Import in another component:

```
import Greeting from './Greeting.jsx';
```

3. Use it like an HTML tag:

```
<Greeting name="Sonsoles" />
```

→ Output:

Hello, Sonsoles

Rendering multiple components with .map()

```
import Greeting from './Greeting.jsx';

function App() {
  const names = ["Toast", "Naranja",
                 "Ramona", "Kinsky"];

  return names.map((catName, i) =>
    <Greeting key={i} name={catName} />
  );
}
export default App;
```

↑
key — required when rendering lists, helps React track which item changed

→ Renders one Greeting component per name

Hello, Toast
Hello, Naranja
Hello, Ramona
Hello, Kinsky

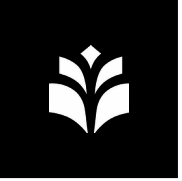
How does it all fit together

SETUP CHAIN

Node.js — runtime environment that runs JavaScript outside the browser
↓ (comes with npm)
npm — package manager that installs libraries (React, Vite, etc.)
↓ (installs)
Vite — development server with hot reloading + production build tool
↓ (uses Babel internally)
React + Babel

DEVELOPMENT CHAIN

WE WRITE React components in JSX (.jsx files)
↓
Vite serves the file
↓
Babel transpiles JSX → vanilla JavaScript
↓
BROWSER RUNS vanilla JavaScript



FROM UI PROBLEMS TO COMPONENT THINKING

- Why do traditional DOM approaches become difficult as apps grow?
- How can a UI be decomposed into reusable components?
- How does information move in a component-based application?

REACT BASICS

Why React?

- Why React, not plain JavaScript? What are React's main characteristics?

Components and JSX

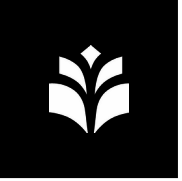
- What is a React component, and how is it defined? What is JSX?

Starting a React project

- How to set up a React project? How to create and use components?

React Developer Tools — inspecting your React application

- How to install React Developer Tools in the browser?
- How to see the component tree of my React application?

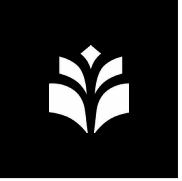


Installing React Developer Tools

A browser extension that adds React inspection to your browser's developer tools. E.g. from Firefox Add-ons or Chrome Web Store search "React Developer Tools"

Once installed:

→ Open your React app in the browser
→ Press F12 to open DevTools
→ Look for the new "Components" tab



Inspecting the component tree

Components tab shows:

→ Your React application as a component tree

→ Which components are rendered on the page

→ Hover over a component → highlights in the browser

Using Dev Tools to understand your app

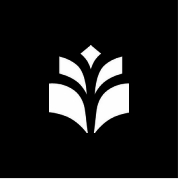
Click any component in the tree:

→ See what it received (props — covered in React II)
→ See its current internal data (state — covered in React II)
→ Edit values live to test changes

When something does not look right:

1. Open Developer Tools (F12)
2. Find the component in the Components tab
3. Check what it received and what it rendered
4. Compare to what you expected

Before asking for help — check Developer Tools first.



Summary

- when apps grow, DOM grows unmanageable: scattered state, code duplication
- to decompose UI find repeating patterns, identify props and state
- in React, information flows one way: data down via props, events up via callbacks
- JS uses manual DOM updates with getElementById, React updates DOM by how UI is described
- main characteristics of React include virtual DOM, reusable components and one-way binding
- React component is a function that accepts props and returns JSX
- component is defined in .jsx, exported with export default
- JSX = JS + XML syntax, transpiled by Babel
- to set up a React project use Node.js → npm create vite@latest → npm install → npm run dev
- to create and use components ureate .jsx file → export default → import → use as <Tag />, to render multiple components: array.map()
- React Developer Tools can be installed from browsers extensions/add-ons
- By opening DevTools (F12) the component tree can be seen on Components tab